

Governance by Construction for Generalist Agents

Segev Shlomov* IBM Haifa, Israel Segev.Shlomov1@ibm.com	Iftach Shoham IBM Haifa, Israel Iftach.Shoham@ibm.com	Alon Oved IBM Haifa, Israel alon.oved@ibm.com	Ido Levy IBM Haifa, Israel Ido.Levy1@ibm.com	Sami Marreed IBM Haifa, Israel sami.marreed@ibm.com
Harold Ship IBM Haifa, Israel harold@il.ibm.com	Offer Akrabi IBM Haifa, Israel offer.akerabi@il.ibm.com	Sergey Zeltyn IBM Haifa, Israel sergeyz@il.ibm.com	Avi Yaeli IBM Haifa, Israel aviy@il.ibm.com	Nir Mashkif IBM Haifa, Israel nirm@il.ibm.com

Abstract

Enterprise agents are increasingly expected to operate autonomously across tools and interfaces, yet production deployments require governance by construction. Systems must specify which actions are allowed, when human oversight is required, and what information may be exposed, without rebuilding the agent for each domain. This demo presents CUGA's policy system, a modular policy-as-code layer that composes with a generalist LLM agent to deliver predictable, auditable, and compliance-aware behavior in compound workflows without model fine-tuning. We present a runtime governance architecture that enforces policy interventions at every critical stage of execution. Rather than passively constraining behavior, policies intercept the agent at five structural checkpoints: upstream of planning (Intent Guard), within the system prompt to steer reasoning (Playbook), at the tool-call boundary to enforce proper usage (Tool Guide), outside the reasoning loop as a Human-in-the-Loop gate for high-risk actions (Tool Approvals), and at the output stage to filter and structure the final response (Output Formatter). Together, these stages embed governance continuously across the agent's execution pipeline rather than treating it as an afterthought. Using a healthcare scenario and a multi-layered enforcement intervention, the demo shows dynamic playbook injection for structured tool-sequence enforcement, intent guards that block malicious or accidental harmful requests, and human-in-the-loop tool approval checkpoints for potentially destructive actions. The artifact illustrates how typed governance primitives enable faster, safer deployment of enterprise agentic systems while improving policy adherence and execution consistency.

CCS Concepts

• Computing methodologies → Intelligent agents.

Keywords

Generalist Agent, Computer Using Agent, Governance, Policy System, LLM Agent

*Corresponding author email: segev.shlomov1@ibm.com.



This work is licensed under a Creative Commons Attribution 4.0 International License.
CAIS '26, San Jose, CA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2415-2/26/05

<https://doi.org/10.1145/3786335.3813192>

ACM Reference Format:

Segev Shlomov, Iftach Shoham, Alon Oved, Ido Levy, Sami Marreed, Harold Ship, Offer Akrabi, Sergey Zeltyn, Avi Yaeli, and Nir Mashkif. 2026. Governance by Construction for Generalist Agents. In *ACM Conference on AI and Agentic Systems (CAIS '26)*, May 26–29, 2026, San Jose, CA, USA. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3786335.3813192>

1 Introduction

LLM-based agents are increasingly adapted to perform complex, multi-step tasks across enterprise software environments, extending earlier work on conversational RPA, next-action recommendation, and process-aware automation [12, 21, 24]. Unlike static chatbots, modern agents integrate planning, internal and external tool-use, memory, and iterative reasoning to execute compound workflows across heterogeneous systems. Recent advances in tool-augmented and computer-using agents enable interaction with APIs, databases, and user interfaces, allowing agents to autonomously retrieve data, modify records and trigger communications. Although this flexibility enables generalization, prior work on LLM automation agents and web-agent benchmarks [23] suggests [1, 7, 9, 10, 15, 22] that it also introduces unpredictability: agents may hallucinate facts, misuse tools, violate procedural constraints, behave inconsistently, or expose sensitive information [8, 14, 17]. In enterprise settings, such failures are not merely quality degradations; they can result in compliance violations, data leakage, financial impacts, or reputational damage.

Many current governance strategies rely on prompt-engineering techniques, instruction stuffing, constraint injection, and post-hoc validation [4, 19, 26]. While these approaches can shape model behavior in controlled scenarios, they exhibit several structural limitations. Behavioral constraints are tightly coupled to prompt structure and can become brittle as policies evolve; governance logic often becomes duplicated across multi-agent deployments; enforcement decisions are delegated to model reasoning and therefore difficult to audit; and independently implemented guardrails may produce inconsistent outcomes without principled conflict resolution. More structured approaches encode operating procedures through role specialization or guard code [5, 26], but typically require substantial architectural commitment or domain-specific reconfiguration when procedures change. Parant's [13] guideline-driven alignment layers dynamically retrieve and inject natural-language rules and tool use at inference time but remain focused on local, chat-level compliance. Skill-based frameworks such as Claude's skill system provide reusable behavioral templates

applied during execution. Both approaches still rely on the model to interpret and follow instructions, leaving policy adherence inherently probabilistic and vulnerable. As enterprise adoption accelerates, governance must evolve beyond prompt-level heuristics toward policy-by-construction: explicit, typed, runtime-enforced control primitives that operate independently of the model itself.

In this work, we present the implementation of our vision for trustworthy, enterprise-ready generalist agents, building on prior work on automation trust, web-agent bottlenecks, and safety-oriented agent evaluation, and demonstrate it through the open-source CUGA agent [2, 11, 16]. We introduce the *CUGA policy system*, a modular policy-as-code layer built into CUGA without requiring model fine-tuning. The framework introduces typed governance primitives that constrain intent recognition, planning, tool invocation, human approval requirements, and output formatting *at runtime*, enforcing consistency and governance. Policies are matched using lightweight triggers (keywords and embedding similarity), which mitigates reliance on probabilistic methods, resolved through explicit conflict handling, and produces structured explanation traces for observability and quality assurance. It offers no need for architectural changes, complies using open-source models (e.g., GPT-OSS-120B), for on-prem deployment, does not affect or modify the agent’s output, and showcases build-in observability. By externalizing governance into composable runtime policies, the system enables predictable, auditable, and compliance-aware behavior across compound workflows while preserving the flexibility of general-purpose LLM agents ensuring compliance and trustworthiness in enterprise-ready agents. We demonstrate our approach through an end-to-end enterprise workflow that highlights runtime policy enforcement. The demonstration centers on two representative scenarios: a healthcare assistance workflow and a multi-layered enforcement intervention.

The demo showcases four out of five modular governance primitives defined as markdown policies at by interference at difference execution checkpoints. The Playbook enforces structured multi-step execution by dynamically associating requests with predefined tool sequences and enterprise-specific constraints, as the Tool Guide enriches tool description. The Intent Guard prevents malicious or accidental harmful actions by intercepting and blocking restricted intents prior to tool execution. The Tool Approval mechanism introduces a *human-in-the-loop* (HITL) checkpoint that pauses the execution graph and requires explicit confirmation before potentially destructive actions can proceed.

2 Methods Overview

Our CUGA’s policy system strengthens enterprise-ready governance through dynamic interception of different policies throughout the agent’s execution graph via the trigger mechanism in five different points: First, *Intent Guard* sit at the very beginning of the process to immediately block bad requests before the agent even takes action. Second, *Playbook* are seamlessly injected into the system prompt, step-by-step to steer the agent’s planning and influence its reasoning. Third, *Tool guide* update and mutate tool descriptions right before execution to instruct the agent on proper tool usage. Fourth, *Tool approval* act as a critical safeguard outside the reasoning loop to gate execution, pausing the graph to wait for

human confirmation if a risky action is attempted. Finally, *Output formatter* serves as the last intervention point to appropriately filter the final answer before it is returned. This section presents the architectural design principles, data models, and runtime mechanisms underlying each policy category. The emphasis is placed on modularity, extensibility, and seamless integration with the LangGraph execution framework.

2.1 Policy System Architecture

The policy system is implemented as a modular, layered framework consisting of four architectural tiers:

- (1) **Policy Models Layer:** Strongly-typed data models defining policy schemas, triggers, and action semantics.
- (2) **Storage Layer:** Persistent and semantic storage backed by a vector database for similarity-based retrieval.
- (3) **Policy Agent Layer:** Runtime matching and conflict-resolution logic that evaluates policies against execution context.
- (4) **Enactment Layer:** Execution primitives that apply policy decisions within the LangGraph workflow.

This layered design enforces separation of concerns: policy representation is decoupled from storage, matching logic, and execution semantics. Policy evaluation occurs at four semantically meaningful checkpoints: (1) intent analysis, (2) tool preparation, (3) post-code generation, and (4) final response generation.

Trigger System. All policy types, except for Tool Approval, rely on a *configurable trigger mechanism*. Triggers are defined as discriminated union types, enabling flexible matching strategies within a unified interface. Supported trigger mechanisms include:

- **Natural Language:** Semantic similarity matching via embedding-based retrieval with configurable similarity thresholds.
- **Keyword:** Exact or fuzzy keyword matching with logical composition (AND/OR) and case sensitivity controls.
- **Application:** Contextual matching based on the active application domain.
- **State:** Evaluation against structured agent state using equality, containment, or regular expression operators.
- **Tool:** Detection of tool usage at specific execution stages (pre- or post-invocation).

Triggers may target different contextual fields, including inferred intent, intermediate sub-tasks, and final agent responses. This flexibility enables policies to intervene at different abstraction levels of the execution life-cycle. Policy persistence and semantic retrieval are implemented using Milvus as a vector database backend. Embeddings are generated using either API or a local-based encoder [20].

2.2 Intent Guard

Intent Guard policies enforce hard constraints by intercepting user intent outside the agent reasoning loop, before the agent acts. Their primary purpose is early-stage blocking of unauthorized or restricted operations. Upon activation, an Intent Guard terminates execution immediately, this early termination mechanism prevents downstream reasoning or unwanted tool invocation. Intent Guard trigger evaluation follows a two-phase process: **(1) Deterministic:** keyword-based triggers are evaluated first. Intent Guards are

prioritized over other policy types to guarantee that blocking constraints supersede advisory policies. (2) **conflict resolution**: for natural-language triggers, multiple policies may satisfy the similarity threshold. In such cases, an LLM-based structured reasoning step selects the most appropriate policy. The model outputs a selected index, confidence score, and justification.

2.3 Playbook

Playbook policies provide structured workflow guidance for complex tasks. Rather than blocking execution, they shape agent planning behavior by injecting step-by-step instructions. This is particularly valuable as it delivers precise, targeted instructions without inflating the prompt with excessive tokens. This causes the agent to follow instructions and consistently comply to the user’s tasks. The Playbook consists of markdown-formatted guidance content, an optional ordered list of steps, optional expected outcomes per step, optional tool constraints per step, trigger definitions, and priority. This structured representation enables fine-grained orchestration and verification of multi-step workflows.

2.4 Tool Approval

Tool Approval policies enforce *HITL* oversight by requiring explicit confirmation before executing sensitive tools. Unlike trigger-based policies, they are evaluated after code generation, allowing inspection of the actual tools the agent intends to invoke in runtime.

After code generation, the system scans the code for tool invocations. If a matching tool is detected, execution pauses and enters a waiting state. The agent resumes only after explicit approval (or automatic approval, if configured). If multiple policies match, the highest-priority policy is applied. This mechanism is particularly useful for sensitive operations such as data modifications (e.g., database writes and updates) and external API calls, where interactions with third-party services should be reviewed before execution.

2.5 Tool Guide

Tool Guide policies augment tool descriptions with contextual or compliance-related guidance. Multiple Tool Guide policies may apply simultaneously, as they are cumulative rather than mutually exclusive. At runtime, tool definitions are deep-copied and enriched with the configured guidance, which may be appended to the original description. The deep-copy mechanism ensures that modifications remain session-scoped and do not permanently alter the underlying tool metadata.

2.6 Output Formatter

Output Formatter policies transform the final response into a structured format. They support three modes: (1) a predefined template verbatim, (2) restructuring the response as formatted Markdown, or (3) extracting structured data according to a specified JSON schema. Trigger evaluation considers both the user input and the generated response, enabling context-aware formatting decisions without relying on the agent’s capabilities.

3 System Demonstration: CUGA’s Policy System

The demo video¹ presents the live CUGA interface, illustrating how governance policies are enforced through targeted interventions in the agent execution graph. Rather than embedding complex enterprise constraints directly in prompts, CUGA applies policies dynamically during agent execution, enabling safer and more reliable task handling (for more information see Section 2). Highlighting the importance of governance by construction, the walkthrough presents two representative scenarios designed to showcase CUGA’s ability to guide agent behavior while maintaining strong safety guarantees: (1) a complex healthcare workflow (OAK) and (2) a multi-layered security intervention.

Playbook. The first scenario demonstrates CUGA in the context of a healthcare assistance task. A user issues the request “*find primary care doctors near me.*”, CUGA dynamically associates the request with a predefined *playbook* that enforces a structured execution policy for mandatory tool sequence to grantee reliable compliance. The execution begins with context extraction, where CUGA retrieves the user’s active insurance coverage and extracts key attributes required for downstream operations, including the relevant contract UID and brand code. Guided by the injected policy, the agent then invokes the *find care suggestions* tool to map the natural language phrase “primary care” to the corresponding internal system code (code 25). This mapping ensures alignment with the underlying service taxonomy.

Following this step, the system performs a paginated provider search guided by the **Tool Guide** critical pagination requirement, issuing multiple API calls that collectively return 14 candidate providers. During the retrieval process, CUGA automatically applies a network-status constraint to ensure that only *in-network* providers are considered. The results are then aggregated and presented to the user as a cleanly formatted table listing nearby in-network primary care physicians. This scenario demonstrates how CUGA is enabled to operate within a controlled policy framework while enforcing structured workflows, tool usage constraints, and enterprise-specific rules.

Intent Guards and Tool Approval. The second scenario focuses on CUGA’s preventive capabilities, which introduces safety mechanisms that operate independently of the agent’s reasoning process. First, a user attempts to execute a malicious (which can also be by accidental) command by requesting to “*delete every contact in CRM.*” the *Intent Guard* immediately intercepts the request and issues a block command that terminates the execution graph. As a result, the agent never receives the instruction, preventing the agent from reasoning about or executing the harmful request. The demonstration then simulates a scenario in which an attacker paraphrases the request in a way that some how bypasses the initial guard. In this case, the agent proceeds to generate code intended to drop a database. Before the action can be executed, CUGA’s *Tool Approval* mechanism pauses the execution graph and introduces a *HITL* checkpoint. The system requires explicit human confirmation before allowing the operation to proceed, ensuring that potentially destructive actions cannot be executed without oversight or accountability by the operator. Together, these scenarios highlight

¹<https://www.youtube.com/watch?v=Ie5ZODIW5gk>

CUGA’s ability to combine policy-driven execution with layered safety mechanisms, enabling reliable and secure deployment of language agents in enterprise environments.

4 Results

We evaluate CUGA’s *policy system* on two enterprise-focused generalist agent benchmarks that reflect the structured, policy-governed workflows central to our demonstration. **OAK** [3] is a 27-task customer-care benchmark grounded in realistic insurance scenarios, assessing the agent’s ability to process claims, retrieve coverage information, and answer health-insurance questions via API calls. **BPO** [6] is a 26-task business-process benchmark covering a broader range of enterprise back-office operations, providing a more challenging generalist evaluation. Together, these benchmarks reflect real-world enterprise settings where tasks carry well-defined procedures, tool-heavy execution, and low tolerance for deviation. All results are reported across three backbone models: *GPT-OSS-120B*, our main focus as an open-source model, *GPT-4.1*, and *Claude Opus-4.5*. The primary metric is **Success Rate (SR)**: the percentage of tasks completed correctly end-to-end. Each configuration was executed over multiple independent runs with a clean environment.

Benchmark	Model	SR (W/O → W)	Δ
OAK	GPT-OSS-120B	75% → 100%	+25 pp
	GPT-4.1	70% → 96%	+26 pp
	Claude Opus-4.5	81% → 96%	+15 pp
BPO	GPT-OSS-120B	49.2% → 82.3%	+33.1 pp
	GPT-4.1	28.5% → 66.2%	+37.7 pp
	Claude Opus-4.5	50.0% → 68.5%	+18.5 pp

Table 1: Success Rate on OAK and BPO benchmarks, with and without CUGA’s policy system, across three backbone models. W/O = without policy system; W = with policy system.

The results in Table 1 show consistent and substantial gains across both benchmarks and all three models, ranging from +15 to +37.7 percentage points. On OAK Bench, the policy system drives GPT-OSS-120B to a perfect 100% SR, with GPT-4.1 and Claude Opus-4.5 reaching 96%. This is particularly evident in complex, multi-step tasks where, without the policy system, the agent deviates from required procedures or invokes tools out of sequence, mirroring how the demo’s workflow becomes a structured, reliable process only once the Playbook is active. On the more demanding BPO benchmark, absolute gains are even larger: GPT-4.1 improves by +37.7 pp despite its lower baseline, suggesting the policy system is especially impactful where the model’s native instruction-following is weakest. Taken together, these results confirm that *governance-by-construction* is both effective and model-agnostic, achieving strong and consistent performance without reliance on frontier LLMs. Runtime governance does, however, introduce token overhead, an inherent consequence of replacing probabilistic compliance with explicit, auditable instructions (full figures in Section A). In enterprise deployments where a single policy violation may constitute a compliance breach or trigger a destructive operation, this tradeoff is well justified.

4.1 BPO Policy Ablation

To isolate the contribution of individual governance primitives, we ran CUGA on BPO under a controlled ablation: *no policies*, *2 policies*, and *5 policies*, using GPT-OSS-120B. Each configuration was executed over three independent runs with a clean vector store (Milvus) to eliminate carryover effects. Policies were matched at runtime via lightweight triggers and injected into the agent loop without model fine-tuning. Success rate improves monotonically: 46.2% (12.0/26) with no policies → 71.8% with 2 policies → 78.2% with 5 policies (+32.0 pp overall). Each step corresponds directly to the governance primitives introduced in Section 2:

(1) **Capability Boundaries (Intent Guard)**. Without policies, the agent attempted to answer unsupported queries by requesting irrelevant identifiers or invoking unrelated APIs. An explicit capability-boundary policy caused it to correctly decline out-of-scope questions, converting multiple tasks from 0/3 to 3/3 (e.g., Tasks 16, 19, 21–23).

(2) **Tool Guide**. Several failures stemmed from calling known-unreliable endpoints (returning 503 errors or malformed schemas). Augmenting tool descriptions with warning guidance caused the agent to re-route to stable APIs, stabilizing previously flaky tasks (e.g., Tasks 12, 14, 18).

(3) **Playbook (Structured Reasoning Constraint)**. For multi-metric questions, the agent previously shortcut to summary endpoints and returned incorrect counts. A multi-API reasoning playbook enforced granular tool selection, resolving systematic count errors (e.g., Task 9) and restoring full correctness. Full per-task results and evaluation sub-metrics are provided in Section A of the supplementary material.

5 Conclusion - Our Vision

As LLM-based agents become increasingly prevalent across industry, their autonomous operation in diverse and sensitive environments introduces critical challenges around safety, compliance, and predictability. Recent incidents involving data and privacy leakage in computer-using agents underscore the urgency of addressing these risks, particularly as deployment moves beyond controlled research settings into production environments handling real users and sensitive data. Our vision is grounded in the belief that capable agents and trustworthy agents should be one and the same. As generalist computer-using agents progress toward production readiness, a principled governance layer becomes not a limitation, but a prerequisite for responsible deployment. This work demonstrates one realization of that vision through CUGA’s policy system, which provides structured mechanisms to constrain unwanted and unpredictable agent behavior without sacrificing generality or performance, but the other way around. CUGA as an open-source, enterprise-oriented agent supporting on-premises deployment, reflects the operational realities of organizations that cannot rely on third-party cloud infrastructure for sensitive workloads. Its strong benchmark performance across diverse agent evaluation settings (e.g., AppWorld [18], WebArena [25], BPO-TA [16]) and by our results in Section 4, further validates it as a credible and production-relevant baseline. These properties, combined with lessons drawn from prior work and companies’ use cases, informed both our design choices and our understanding of where policy enforcement is

most needed. Ultimately, our policy system is a means to a broader end: the development of generalist agents that organizations can deploy with confidence.

References

- [1] Zhaorun Chen, Mintong Kang, and Bo Li. 2025. Shieldagent: Shielding agents via verifiable safety policy reasoning. *arXiv preprint arXiv:2503.22738* (2025).
- [2] CUGA Project. 2026. CUGA: Computer-Using Generalist Agent. <https://github.com/cuga-project/cuga-agent>. Accessed: 2026-04-27.
- [3] CUGA Project. 2026. OAK Bench: Customer-Care Benchmark for Insurance Workflows. <https://github.com/cuga-project/oak-bench>. Accessed: 2026-04-27.
- [4] Suyash Gaurav, Jukka Heikkonen, and Jatin Chaudhary. 2025. Governance-as-a-service: A multi-agent framework for ai system compliance and policy enforcement. *arXiv preprint arXiv:2508.18765* (2025).
- [5] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiaowu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. 2023. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The twelfth international conference on learning representations*.
- [6] IBM Research. 2026. BPO-Bench: Business Process Operations Benchmark. <https://huggingface.co/datasets/ibm-research/BPO-Bench>. Accessed: 2026-04-27.
- [7] Changyue Jiang, Xudong Pan, and Min Yang. 2025. Think twice before you act: Enhancing agent behavioral safety with thought correction. *arXiv preprint arXiv:2505.11063* (2025).
- [8] Ido Levy, Ben Wiesel, Sami Marreed, Alon Oved, Avi Yaeli, and Segev Shlomov. 2024. St-webagentbench: A benchmark for evaluating safety and trustworthiness in web agents. *arXiv preprint arXiv:2410.06703* (2024).
- [9] Hanjun Luo, Shenyu Dai, Chiming Ni, Xinfeng Li, Guibin Zhang, Kun Wang, Tongliang Liu, and Hanan Salam. 2025. Agentauditor: Human-level safety and security evaluation for llm agents. *arXiv preprint arXiv:2506.00641* (2025).
- [10] Xingjun Ma, Yifeng Gao, Yixu Wang, Ruofan Wang, Xin Wang, Ye Sun, Yifan Ding, Hengyuan Xu, Yunhao Chen, Yunhan Zhao, et al. 2026. Safety at scale: A comprehensive survey of large model and agent safety. *Foundations and Trends in Privacy and Security* 8, 3-4 (2026), 1–240.
- [11] Sami Marreed, Alon Oved, Avi Yaeli, Segev Shlomov, Ido Levy, Offer Akrabi, Aviad Sela, Asaf Adi, and Nir Mashkif. 2025. Towards enterprise-ready computer using generalist agent. *arXiv preprint arXiv:2503.01861* (2025).
- [12] Alon Oved, Segev Shlomov, Sergey Zeltyn, Nir Mashkif, and Avi Yaeli. 2025. SNAP: semantic stories for next activity prediction. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 28871–28877.
- [13] Parlant. 2025. Parlant: Conversational Control Layer for Customer-Facing LLM Agents. <https://parlant.io/> and <https://github.com/emcie-co/parlant>. Accessed March 2026.
- [14] Sivan Schwartz, Avi Yaeli, and Segev Shlomov. 2023. Enhancing trust in LLM-based AI automation agents: New considerations and future challenges. *arXiv preprint arXiv:2308.05391* (2023).
- [15] Yucheng Shi, Wenhao Yu, Jingyuan Huang, Wenlin Yao, Wenhui Chen, and Ninghao Liu. 2025. Towards trustworthy gui agents: A survey. *arXiv preprint arXiv:2503.23434* (2025).
- [16] Segev Shlomov, Alon Oved, Sami Marreed, Ido Levy, Offer Akrabi, Avi Yaeli, Łukasz Strąk, Elizabeth Koumpan, Yinon Goldshtein, Eilam Shapira, et al. 2026. From Benchmarks to Business Impact: Deploying IBM Generalist Agent in Enterprise Production. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 40423–40431.
- [17] Segev Shlomov, Ben Wiesel, Aviad Sela, Ido Levy, Liane Galanti, and Roy Abitbol. 2025. From Grounding to Planning: Benchmarking Bottlenecks in Web Agents. In *ECAI 2025 – 28th European Conference on Artificial Intelligence (Frontiers in Artificial Intelligence and Applications, Vol. 413)*. IOS Press, 4815–4822. [arXiv:2409.01927 \[cs.AI\]](https://arxiv.org/abs/2409.01927)
- [18] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024. Appworld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 16022–16076.
- [19] Lillian Tsai and Eugene Bagdasarian. 2025. Contextual agent security: A policy for every purpose. In *Proceedings of the 2025 Workshop on Hot Topics in Operating Systems*. 8–17.
- [20] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Advances in neural information processing systems* 33 (2020), 5776–5788.
- [21] Avi Yaeli, Segev Shlomov, Alon Oved, Sergey Zeltyn, and Nir Mashkif. 2022. Recommending next best skill in conversational robotic process automation. In *International Conference on Business Process Management*. Springer, 215–230.
- [22] Xiao Yang, Jiawei Chen, Jun Luo, Zhengwei Fang, Yinpeng Dong, Hang Su, and Jun Zhu. 2025. Mla-trust: Benchmarking trustworthiness of multimodal llm agents in gui environments. *arXiv preprint arXiv:2506.01616* (2025).
- [23] Zonghao Ying, Yangguang Shao, Jianle Gan, Gan Xu, Wenxin Zhang, Quanchen Zou, Junzheng Shi, Zhenfei Yin, Mingchuan Zhang, Aishan Liu, et al. 2025. Securewebarena: A holistic security evaluation benchmark for llm-based web agents. *arXiv preprint arXiv:2510.10073* (2025).
- [24] Sergey Zeltyn, Segev Shlomov, Avi Yaeli, and Alon Oved. 2022. Prescriptive process monitoring in intelligent process automation with chatbot orchestration. *arXiv preprint arXiv:2212.06564* (2022).
- [25] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. 2023. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854* (2023).
- [26] Naama Zwerdling, David Boaz, Ella Rabinovich, Guy Uziel, David Amid, and Ateret Anaby Tavor. 2025. Towards Enforcing Company Policy Adherence in Agentic Workflows. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*. 595–606.

A Ablation Study: BPO Benchmark

This appendix provides the complete experimental results for the CUGA policy evaluation on the BPO benchmark, including per-run statistics, all metric breakdowns, per-task pass rates, and descriptions of each implemented policy.

A.1 Aggregate Results Across Policy Configurations

Each configuration was evaluated over three independent runs on a clean Milvus database. Table 2 reports per-run scores and summary statistics.

	No Policies	2 Policies	5 Policies
Run 1	11/26 (42.3%)	19/26 (73.1%)	20/26 (76.9%)
Run 2	13/26 (50.0%)	18/26 (69.2%)	20/26 (76.9%)
Run 3	12/26 (46.2%)	19/26 (73.1%)	21/26 (80.8%)
Mean	12.0/26 (46.2%)	18.7/26 (71.8%)	20.3/26 (78.2%)
Std. Dev.	1.0	0.6	0.6
Range	11–13	18–19	20–21

Table 2: Per-run scores for each policy configuration (out of 26 tasks).

Table 3 summarizes all evaluation metrics across configurations, with the total improvement from no-policy to five-policy baseline.

A.2 Per-Task Breakdown

Table 4 reports pass counts (out of 3 runs) for each task across the three configurations, together with the net improvement and task category.

A.3 Policy Descriptions

The five implemented policies are described below in order of application.

Policy #1 – API Capability Boundaries. *Type: Playbook. Trigger: keywords + natural language (threshold 0.65, priority 90).*

This policy teaches the agent to recognise when no available API can answer a query. It enumerates supported and unsupported API capabilities and instructs the agent to decline out-of-scope requests directly, rather than soliciting a requisition ID or invoking irrelevant tools.

Metric	No Policies	2 Policies	5 Policies	Δ Total
Final score	12.0/26 (46.2%)	18.7/26 (71.8%)	20.3/26 (78.2%)	+32.0 pp
Avg. similarity	0.539	0.602	0.643	+0.104
Keyword match	57.5%	73.1%	76.0%	+18.5 pp
LLM judge (avg)	0.517	0.742	0.824	+0.307
LLM judge (binary)	52.6%	78.2%	88.5%	+35.9 pp
API accuracy	50.0%	66.7%	73.1%	+23.1 pp

Table 3: Evaluation metrics by policy configuration.

Task	Category	No Pol.	2 Pol.	5 Pol.	Δ
1	Stable pass	3/3	3/3	3/3	+0
2	Stable pass	3/3	3/3	3/3	+0
3	Stable fail	0/3	0/3	0/3	+0
4	Stable pass	3/3	3/3	3/3	+0
5	Stable pass	3/3	3/3	3/3	+0
6	Stable fail	0/3	0/3	0/3	+0
7	Stable fail	0/3	0/3	0/3	+0
8	Stable pass	3/3	3/3	3/3	+0
9	Improved (Policy #3)	0/3	0/3	3/3	+3
10	Stable pass	3/3	3/3	3/3	+0
11	Stable pass	3/3	3/3	3/3	+0
12	Flaky	2/3	3/3	2/3	+0
13	Stable pass	3/3	3/3	3/3	+0
14	Improved (Policy #2)	2/3	3/3	3/3	+1
15	Stable fail	0/3	0/3	0/3	+0
16	Improved (Policy #1)	0/3	3/3	3/3	+3
17	Stable pass	3/3	3/3	3/3	+0
18	Improved (Policy #2)	2/3	3/3	3/3	+1
19	Improved (Policy #1)	0/3	3/3	3/3	+3
20	Stable pass	3/3	3/3	3/3	+0
21	Improved (Policy #1)	0/3	3/3	3/3	+3
22	Improved (Policy #1)	0/3	3/3	3/3	+3
23	Improved (Policy #1)	0/3	3/3	3/3	+3
24	Improved (Policy #1/#2, partial)	0/3	1/3	2/3	+2
25	Stable fail	0/3	1/3	0/3	+0
26	Improved (Policy #4)	0/3	0/3	3/3	+3

Table 4: Per-task pass rates across policy configurations.

Failure pattern addressed. Without this policy the agent would request a requisition ID, or call arbitrary APIs, for queries relating to job descriptions, time-to-fill, geography filtering, SLA deadlines, funnel timing, and job-card details — none of which are supported by any API.

Tasks fixed. 16, 19, 21, 22, 23 (all from 0/3 to 3/3).

Policy #2 — Error-Prone Tool Warnings. Type: Tool Guide. Target: 19 error-prone tools (warning prepended to each description).

This policy prepends a warning to the descriptions of the 19 known-unreliable tools (those returning 503 errors, schema violations, or type mismatches). It steers the agent towards the 13 reliable core tools and teaches graceful recovery when an error tool is nonetheless invoked.

Failure pattern addressed. The agent would call tools such as `funnel_status` (503 error), `model_registry` (incorrect data), or

`source_recommendation_summary` (incomplete shortcut) instead of the correct granular APIs.

Tasks fixed. 12, 14, 18 (stabilised from 2/3 to 3/3); 24 (0/3 to 1–2/3, partial improvement).

Policy #3 — Multi-API Reasoning. Type: Playbook. Trigger: keywords + natural language (threshold 0.65, priority 80).

This policy instructs the agent to call multiple specific APIs for multi-metric questions rather than relying on a single summary endpoint. It provides an explicit mapping from question type to the correct tool and clarifies the distinction between “total requisitions used for computation” (definitions-and-methodology) and “similar requisitions analysed” (metadata-and-timeframe).

Failure pattern addressed. The agent would use the summary shortcut tool for multi-metric queries, or confuse which endpoint returns the requisition count.

Tasks fixed. 9 (0/3 to 3/3; agent now correctly returns 1047 from definitions-and-methodology rather than 40 from metadata-and-timeframe).

Policy #4 — Average vs. Total Calculations. Type: Playbook. Trigger: keywords + natural language (threshold 0.65, priority 70).

This policy teaches the agent that when a user asks for “average” or “typical” values it must compute a per-requisition average by dividing the aggregate total by the count of similar requisitions, rather than returning the raw total.

Failure pattern addressed. The agent would return the total candidate count (2,913) when asked “how many candidates do we usually get?” instead of computing the per-requisition average ($2,913 \div 40 \approx 73$).

Tasks fixed. 26 (0/3 to 3/3).

Policy #5 — Missing Requisition ID vs. Unsupported Query. Type: Playbook. Trigger: keywords + natural language (threshold 0.60, priority 85).

This policy helps the agent distinguish between “I need a requisition ID to answer this” (answerable but missing context) and “this cannot be answered regardless of requisition ID” (unsupported by any API). It reinforces Policy #1 for edge cases where the triggering language differs.

Failure pattern addressed. The agent would request a requisition ID even for queries that no API supports under any circumstances.

Tasks fixed. Overlaps with Policy #1; provides reinforcement for edge cases not caught by Policy #1’s triggers.

A.4 Remaining Failing Tasks

Table 5 documents the six tasks that remain unsolved (or partially unsolved) after all five policies are applied, together with a diagnosis of each failure mode.

B Policy System Evaluations

We presented CUGA’s policy system evaluation in Section 4. Here, we extend that discussion by reporting the token usage associated with each evaluation, providing a quantitative measure of the overhead introduced by our method. Results are shown in Table 6.

Task	Pass rate	Issue
3	0/3	Agent provides incomplete source data (only LinkedIn; misses Dice and GitHub details). Requires richer answer synthesis.
6	0/3	Garbled percentages from the summary tool. May require further tool-description refinement.
7	0/3	LLM judge scores correct agent behaviour as 0. Failure is in the judge, not the agent.
12	2/3	Agent occasionally omits one data-source name. Attributed to LLM non-determinism.
15	0/3	Agent calls too few APIs and misidentifies negative-SLA skills. Complex multi-part question.
24	2/3	Agent sometimes invokes an error tool for an invalid requisition ID. Partially addressed by Policy #2.
25	0/3	Agent calls requisition-details error tool for requisition UZLXBR instead of declining. Edge case not caught by Policy #1 triggers.

Table 5: Remaining failing tasks after five-policy configuration.

Benchmark	Model	Accuracy (W/O $\rightarrow$ W)	Δ	Tokens (W/O $\rightarrow$ W)
OAK	GPT-OSS-120B	75% $\rightarrow$ 100%	+25 pp	26.2k $\rightarrow$ 40k
	GPT-4.1	70% $\rightarrow$ 96%	+26 pp	28.2k $\rightarrow$ 44k
	Claude Opus-4.5	81% $\rightarrow$ 96%	+15 pp	44k $\rightarrow$ 85k
BPO	GPT-OSS-120B	49.2% $\rightarrow$ 82.3%	+33.1 pp	3.7M $\rightarrow$ 6.6M
	GPT-4.1	28.5% $\rightarrow$ 66.2%	+37.7 pp	2.7M $\rightarrow$ 8.8M
	Claude Opus-4.5	50.0% $\rightarrow$ 68.5%	+18.5 pp	6.2M $\rightarrow$ 9.0M

Table 6: Success Rate and Token Usage on OAK and BPO benchmarks, with and without CUGA’s policy system, across three backbone models. W/O = without policy system; W = with policy system.